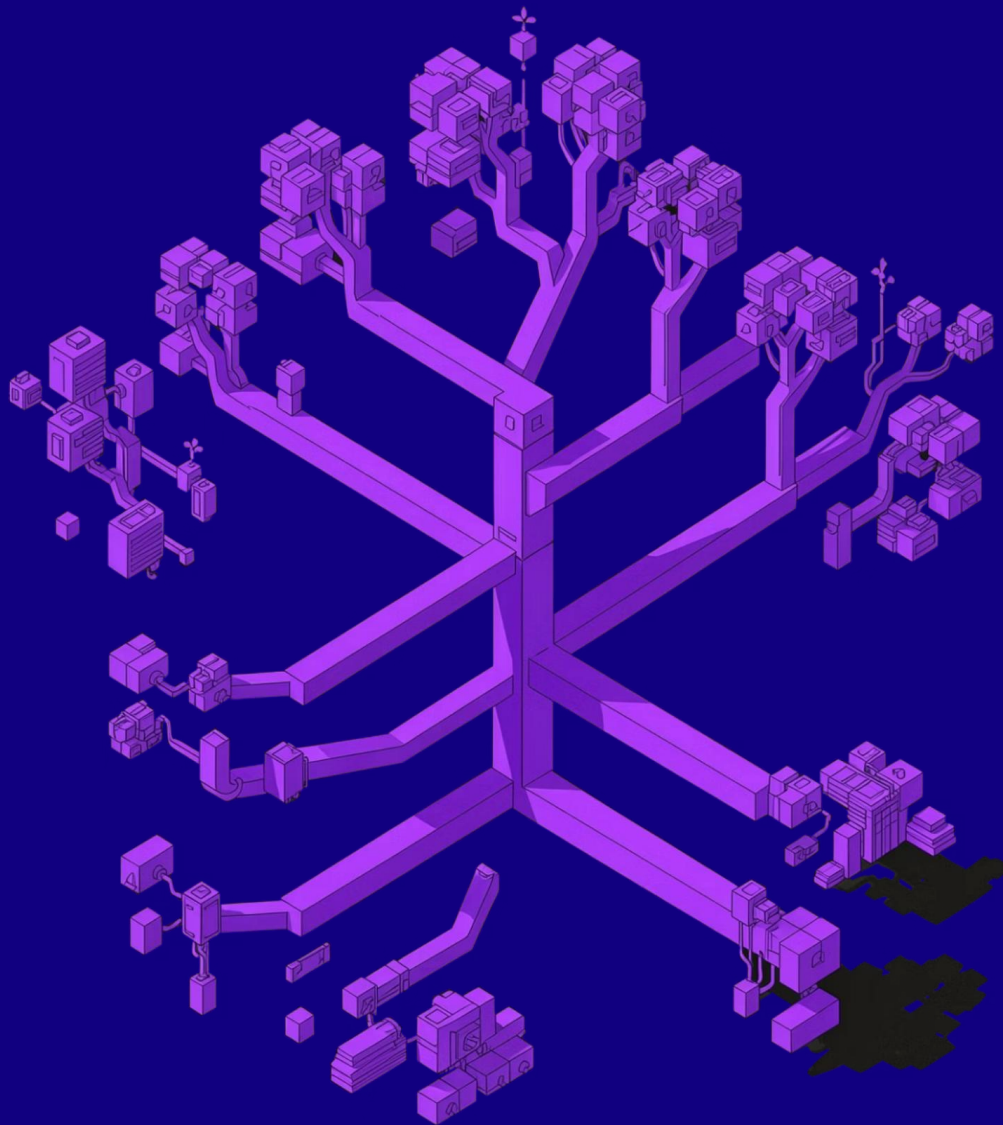




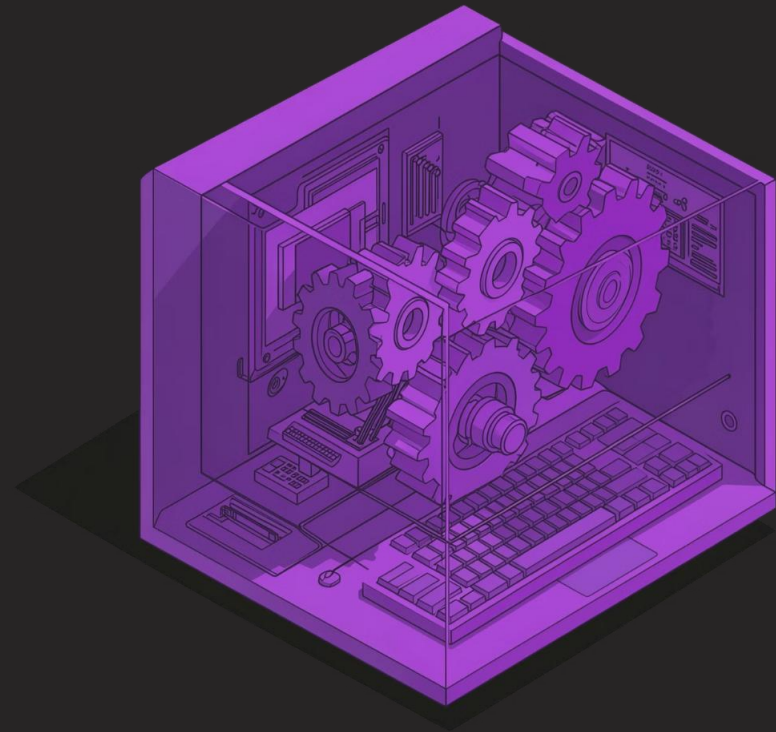
Criação de Processos em C

Entendendo `fork()`, processos pai e filho, concorrência e o conceito de [processador de tarefas](#)

PROGRAMAÇÃO C

SISTEMAS OPERACIONAIS

O que é um Processo?



Um **processo** é um programa em execução. Ele possui seu próprio espaço de memória, estado e contexto gerenciado pelo sistema operacional.

Programa

Código estático em disco

Processo

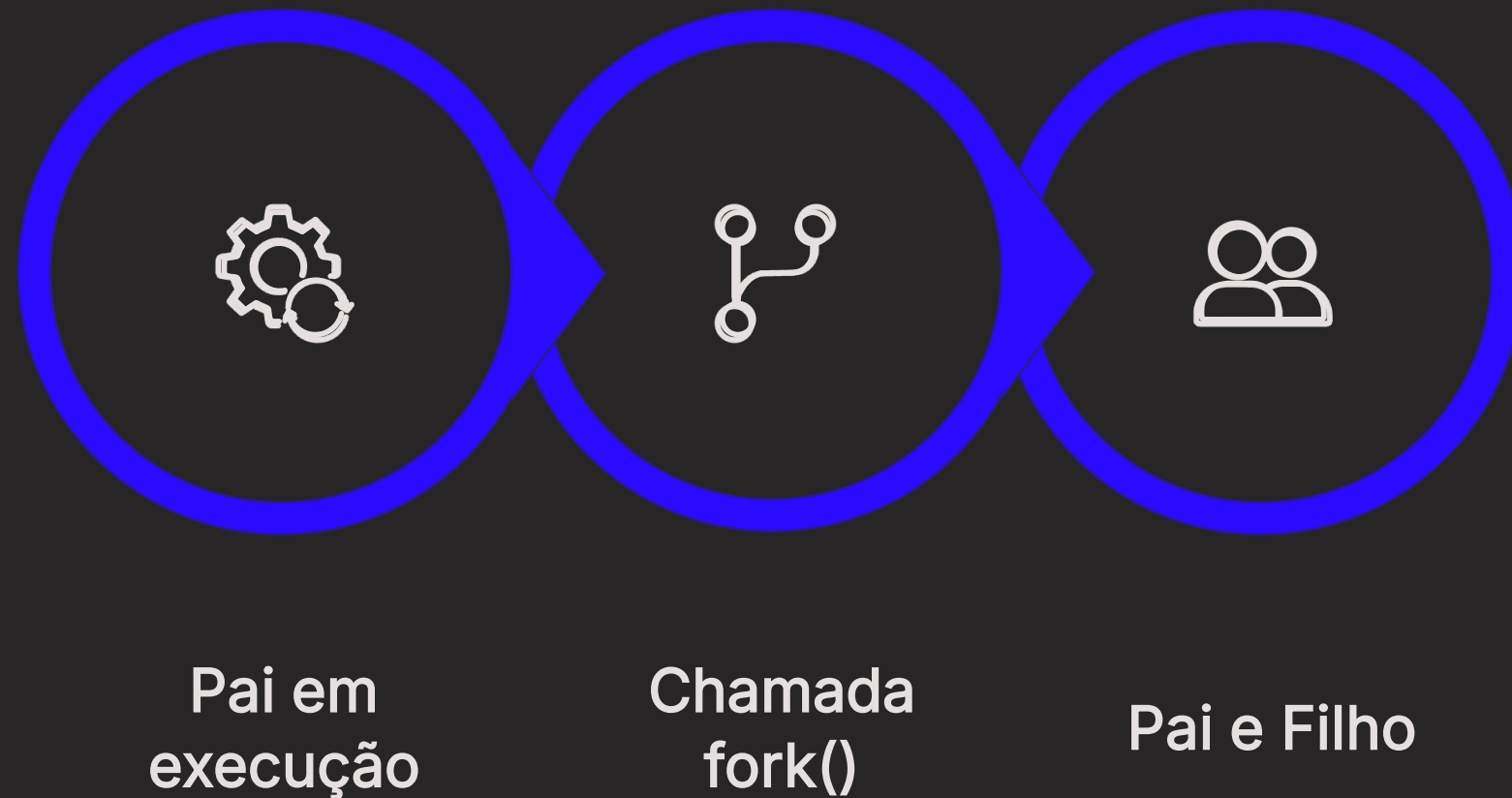
Programa em execução na memória

PID

Identificador único de cada processo

A Função `fork()`

O `fork()` é uma chamada de sistema UNIX que **duplica o processo atual**, criando um processo filho idêntico ao pai.



Após o `fork()`, pai e filho continuam a execução a partir da próxima instrução — cada um com sua própria cópia da memória.

Valores de Retorno do `fork()`

Retorna > 0

Você está no **processo pai**. O valor retornado é o PID do filho.

Retorna $== 0$

Você está no **processo filho**. O filho recebe zero como retorno.

Retorna < 0

Erro! O fork falhou. Verifique permissões e limites do sistema.

```
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();
    if (pid > 0) printf("Sou o PAI, PID do filho: %d\n", pid);
    else if (pid == 0) printf("Sou o FILHO, meu PID: %d\n", getpid());
    else perror("fork falhou");
    return 0;
}
```

Pai e Filho: Executando em Paralelo

Após o `fork()`, os dois processos executam de forma independente e simultânea. A ordem de execução depende do escalonador do SO — nunca é garantida.

→ Memória independente

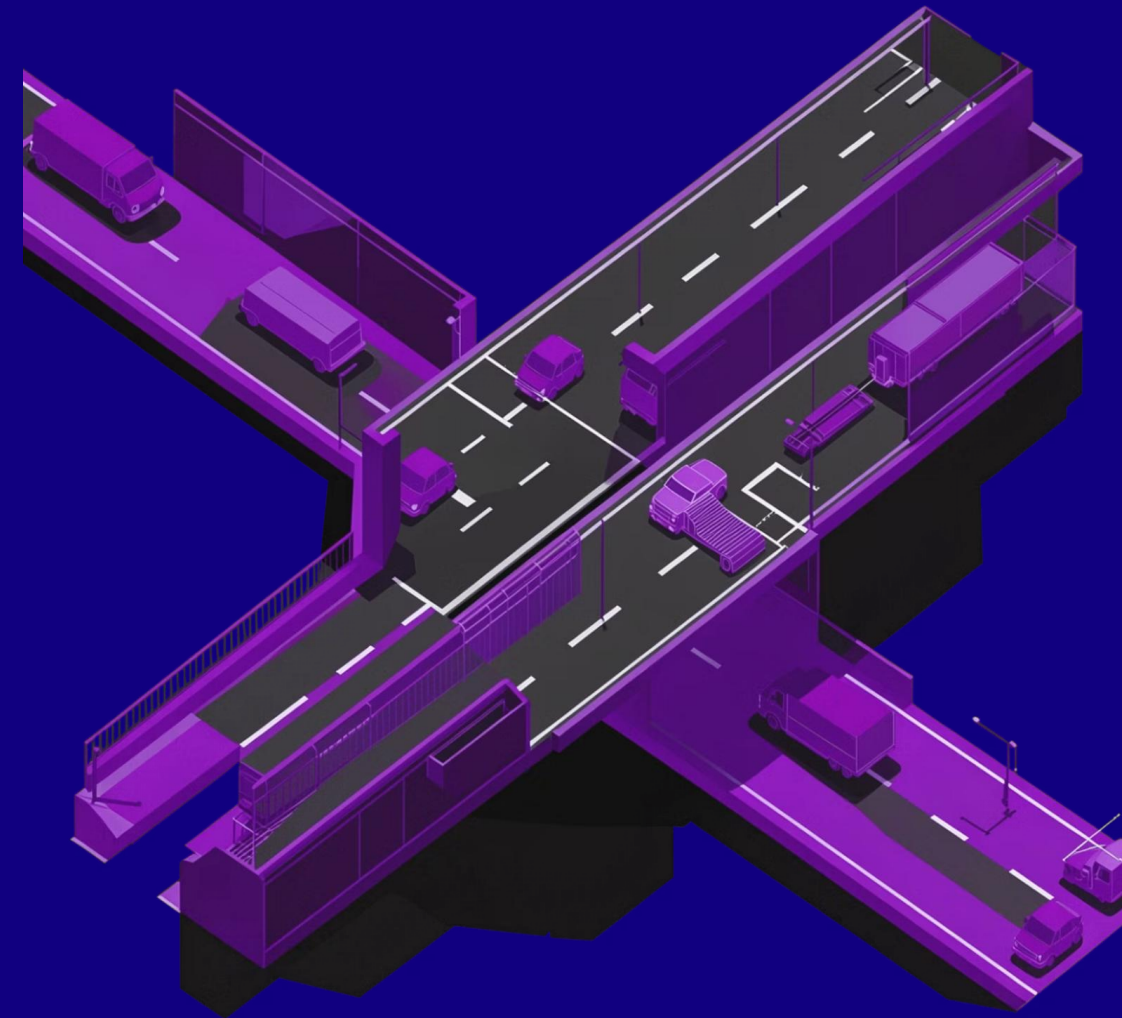
Cada processo tem sua cópia — alterar uma variável no filho não afeta o pai.

→ Herança de descritores

O filho herda arquivos abertos e variáveis do momento do `fork`.

→ `wait()` no pai

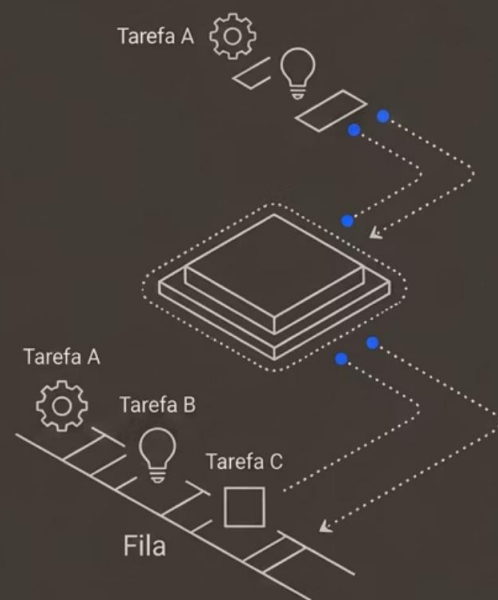
O pai pode usar `wait()` para aguardar o filho terminar e evitar processos zumbi.



Concorrência e Paralelismo

CONCORRÊNCIA

Múltiplas tarefas em andamento. Alternam execução no mesmo núcleo.

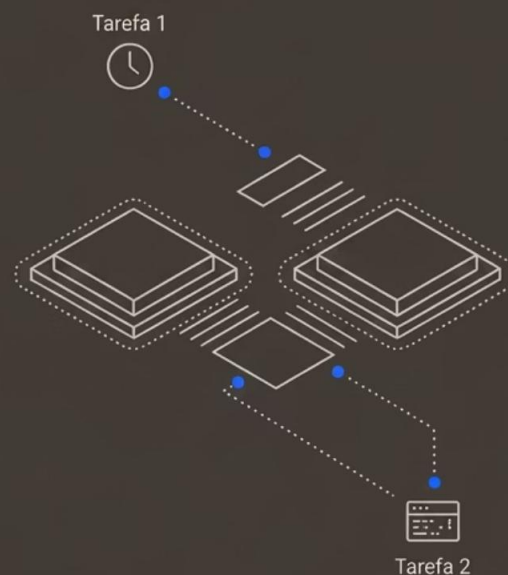


CONCORRÊNCIA

Múltiplas tarefas em andamento. Alternam execução no mesmo núcleo.

PARALELISMO

Múltiplas tarefas executando simultaneamente em núcleos diferentes.



PARALELISMO

Múltiplas tarefas executando simultaneamente em núcleos diferentes.

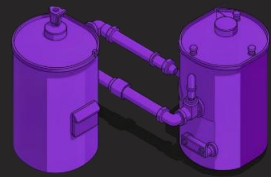
Conceitos Fundamentais

- **Concorrência:** tarefas se revezam no processador — ilusão de simultaneidade
- **Paralelismo:** tarefas executam ao mesmo tempo em múltiplos núcleos
- `fork()` permite explorar ambos os modelos
- O SO decide quando cada processo recebe tempo de CPU

i Em sistemas multicore, processos filhos podem executar em núcleos diferentes simultaneamente.

Comunicação entre Processos

Processos independentes precisam de mecanismos para **trocar informações**. Os dois principais são **pipes** e **sinais**.



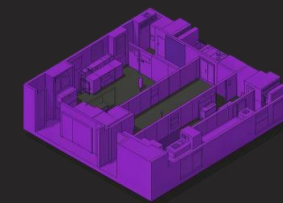
Pipes

Canal unidirecional de dados. O pai escreve, o filho lê (ou vice-versa). Criado com `pipe(fd)`.



Sinais

Notificações assíncronas entre processos. Ex: `SIGTERM`, `SIGKILL`, `SIGUSR1`. Enviados com `kill()`.



Memória Compartilhada

Região de memória acessível por múltiplos processos. Alta performance, mas requer sincronização.

Exemplo: Pipe entre Pai e Filho

O exemplo abaixo demonstra o pai enviando uma mensagem ao filho via pipe:

```
#include <stdio.h>
#include <unistd.h>
#include <string.h>

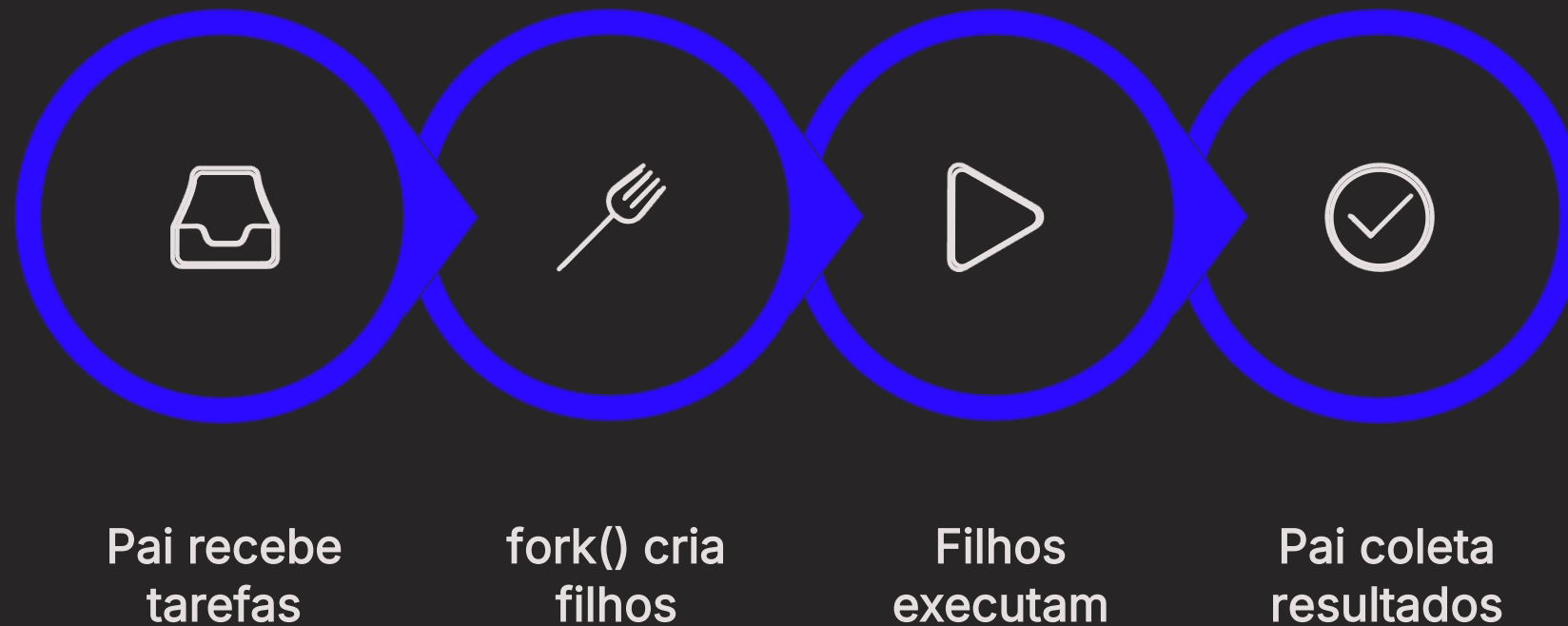
int main() {
    int fd[2];
    char buf[64];
    pipe(fd); // fd[0]=leitura, fd[1]=escrita

    if (fork() == 0) {
        // FILHO: lê do pipe
        close(fd[1]);
        read(fd[0], buf, sizeof(buf));
        printf("Filho recebeu: %s\n", buf);
    } else {
        // PAI: escreve no pipe
        close(fd[0]);
        write(fd[1], "Olá, filho!", 12);
    }
    return 0;
}
```

✔ O pai fecha o lado de leitura e o filho fecha o lado de escrita — boa prática para evitar bloqueios.

Processador de Tarefas com fork()

Um processador de tarefas distribui trabalho entre processos filhos. O pai atua como coordenador — cada filho executa uma tarefa independente.



Fila de Tarefas

Lista de trabalhos a processar

Workers

Processos filhos executando tarefas

Resultado

Pai coleta com `wait()`

 Ideal para processar arquivos, requisições ou cálculos pesados de forma eficiente.

Resumo

01

Processo = Programa em Execução

Todo processo tem PID, memória e estado próprios.

02

fork() Duplica o Processo

Pai recebe PID do filho; filho recebe 0. Ambos executam de forma independente.

03

Comunicação via Pipes e Sinais

Use `pipe()` para dados e `kill()/signal()` para notificações.

04

Processador de Tarefas

Padrão prático: pai delega, filhos executam em paralelo, pai espera com `wait()`.

